Student Name: Husanpreet Kaur **UID: 24BCY70138**
Branch: AIT-CSE (Cyber Security) **Section/Group: 24AIT_KRG2**
Semester: 4 **Date of Performance: 01/04/2026**
Subject Name: Database Management System **Subject Code: 24CSH-298**

WORKSHEET 8

AIM: To design and implement PL/pgSQL procedures and functions and to analyze their execution and usage.

S/W Requirement:
- Database Management System: PostgreSQL
- Database Administration Tool: pgAdmin

OBJECTIVES:

- Create tables and insert data
- Implement stored procedures
- Use IN, OUT, and INOUT parameters
- Create scalar functions
- Create table-returning functions
- Execute and analyze outputs

PROBLEM STATEMENT:

In database systems, reusable logic is essential for efficient data handling. Procedures and functions help reduce redundancy, improve performance, and simplify complex operations.

TABLE STRUCTURE:

```
create table employee_data(
emp_id serial primary key,
emp_name varchar(50),
gender varchar(10),
salary numeric(10,2)
);
```

Data Output Messages Notifications

```
CREATE TABLE

Query returned successfully in 168 msec.
```

```
insert into employee_data(emp_name,gender,salary) values
('Amit','Male',30000),
('Riya','Female',35000),
('John','Male',28000),
('Sneha','Female',40000)
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	emp_id [PK] integer	emp_name character varying (50)	gender character varying (10)	salary numeric (10,2)
1	1	Amit	Male	30000.00
2	2	Riya	Female	35000.00
3	3	John	Male	28000.00
4	4	Sneha	Female	40000.00

1. PROBLEM STATEMENT – Procedure Implementation

Create a procedure to count employees based on gender.

PROGRAM:

```
CREATE OR REPLACE PROCEDURE count_emp(
    IN p_gen VARCHAR(20),
    OUT count_emp INT,
    INOUT status VARCHAR
)
AS
$$
BEGIN
    SELECT COUNT(*) INTO count_emp
    FROM employee_data
    WHERE gender = p_gen;

    status := 'SUCCESS';
END;
$$ LANGUAGE plpgsql;
```

OUTPUT:

```
-- calling
    DO
$$
DECLARE
    GEN VARCHAR(20) := 'Male';
    COUNT_BY_GEN INT;
    STATUS VARCHAR(20) := 'Fail';
BEGIN

    CALL COUNT_EMP(GEN, COUNT_BY_GEN, STATUS);

    RAISE NOTICE 'YOUR GENDER IS % YOUR COUNT IS % AND STATUS IS %',
        GEN, COUNT_BY_GEN, STATUS;

END;
$$;
```

```
Data Output   Messages   Notifications

NOTICE:  YOUR GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS
DO

Query returned successfully in 75 msec.
```

2. PROBLEM STATEMENT – Function Implementation (Scalar)

Create a function to calculate bonus.

PROGRAM:

```
CREATE TABLE employees (
    emp_id INT  PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    salary NUMERIC(10,2) NOT NULL,
    department VARCHAR(50)
);
```

```
INSERT INTO employees (emp_id,emp_name, salary, department) VALUES
(1,'Alice', 50000, 'HR'),
(2,'Bob', 60000, 'IT'),
(3,'Charlie', 55000, 'Finance');
```

```
CREATE OR REPLACE FUNCTION get_bonus(p_emp_id INT)
RETURNS NUMERIC
AS
$$
BEGIN
    RETURN (
        SELECT salary * 0.10
        FROM employees
        WHERE emp_id = p_emp_id
    );
END;
$$ LANGUAGE plpgsql;
```

```
DO
$$
DECLARE
    emp_id INT := 1;
    bonus NUMERIC(10,2);
BEGIN
    bonus := get_bonus(emp_id);

    RAISE NOTICE 'your emp_id is % and your bonus is %', emp_id, bonus;
END;
$$;
```

OUTPUT:

```
Data Output   Messages   Notifications

NOTICE:  your emp_id is 1 and your bonus is 5000.00
DO

Query returned successfully in 77 msec.
```

```
select * from get_bonus(1)
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	get_bonus numeric
1	5000.0000

3. PROBLEM STATEMENT – Function Implementation (Table)

Create a function to return employee details.

```sql
CREATE OR REPLACE FUNCTION get_employees()
RETURNS TABLE(
    id INT,
    name VARCHAR(100)
)
AS
$$
BEGIN
    RETURN QUERY
    SELECT emp_id, emp_name FROM employees;
END;
$$ LANGUAGE plpgsql;
```

OUTPUT:

```sql
--calling part
DO
$$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN SELECT * FROM get_employees()
    LOOP
        RAISE NOTICE 'id: %, name: %', rec.id, rec.name;
    END LOOP;
END;
$$;
```

Data Output Messages Notifications

```
NOTICE:  id: 1, name: Alice
NOTICE:  id: 2, name: Bob
NOTICE:  id: 3, name: Charlie
DO

Query returned successfully in 71 msec.
```

LEARNING OUTCOMES:

1. Understood procedures and parameter modes (IN, OUT, INOUT)
2. Learned how to create and execute functions
3. Implemented scalar and table-returning functions
4. Gained knowledge of PL/pgSQL programming
5. Applied concepts useful in real-world database applications

CONCLUSION:

This experiment demonstrated the implementation of procedures and functions in PostgreSQL. Procedures help perform operations with multiple outputs, while functions efficiently return values and datasets, improving modularity and performance.